PA1 Design Doc

CS 4120: Overview Documentation

Programming Assignment 1: Implementing Lexical Analysis

Authors: Fadi Ismail, Yoseph Endawoke, Charles Liggins, Temi Adebowale
Date: February 4, 2026

---

1. Metadata

- **Assignment:** Programming Assignment 1: Lexer
- **Group members:**
  - Fadi Ismail (fmi4)
  - Yoseph Endawoke (ye38)
  - Charles Liggins (cll258)
  - Temi Adebowale (ta375)
- **How to run:**

```
# Build the compiler
./etac-build

# Run the lexer on a specific .eta file
./etac --lex <filename>.eta

# Run the lexer on all our PA1 test files
for f in eth/tests/pa1/*.eta; do ./etac --lex "$f"; done

# Run all our PA1 tests and check against solutions (print
s FAIL for mismatches)
for f in eth/tests/pa1/*.eta; do ./etac --lex "$f"; done &
```

```
& for f in eth/tests/pa1/*.eta; do base="${f%.eta}"; if [
-f "$base.lexedsol" ]; then diff "$base.lexed" "$base.lexe
dsol" > /dev/null || echo "FAIL: $f"; fi; done && echo "Do
ne"
```

- **Main entry point:** `src/main.rs`
- **Environment tested:** macOS (Darwin 24.6.0), Linux VM, Rust 1.x stable

---

2. Summary

The most challenging aspect of this assignment was handling string and character literal parsing with proper escape sequence support, particularly the `\\x{...}` hex escape format which required careful boundary handling and validation. We chose to use the `logos` crate as our lexer generator framework, which allowed us to declaratively specify token patterns using Rust attributes while providing hooks for complex callback-based parsing of literals.

Key design decisions included:

- Using a callback-based approach for string/character literals rather than pure regex, enabling better error reporting and state management
- Storing integers as `u64` values during lexing (parsed immediately) rather than as strings
- Tracking line and column positions incrementally via `LexerExtras` state to provide accurate error locations
- Stopping lexing on the first error encountered, as subsequent tokens may be unreliable

A known limitation is that our lexer does not attempt error recovery—once an invalid character or malformed literal is encountered, lexing halts. This matches the expected behavior per the specification but means partial output is produced for files with errors.

---

3. Specification

We encountered several ambiguities in the handout that required interpretation:

1. **Line and column numbering:** The specification did not explicitly state whether lines and columns are 0-indexed or 1-indexed. We chose **1-indexed** for both, as this matches conventional editor behavior and the example outputs provided.
2. **Column counting for tabs:** The handout did not specify how tab characters affect column counting. We treat tabs as single characters (column advances by 1), consistent with byte-offset interpretation rather than visual alignment.
3. **Integer overflow:** The specification did not address what happens when an integer literal exceeds the maximum representable value. We parse integers as `u64`, which handles values up to 2^64-1. Overflow during parsing would result in a lexer error, though the Eta specification limits integers to signed 64-bit range in practice.
4. **Empty source files:** We handle empty files gracefully, producing an empty `.lexed` output file with no tokens.
5. **Comments at EOF:** A comment at the end of a file without a trailing newline is handled correctly—the comment is skipped and lexing completes successfully.
6. **Output file placement with `D`:** When `D` is specified, we preserve the relative directory structure of input files in the output directory (e.g., `./etac --lex -D out foo/bar.eta` produces `out/foo/bar.lexed`).

---

4. Design and Implementation

4.1 Architecture

Module layout:

Module	File	Responsibility
CLI	`src/cli.rs`	Command-line argument parsing using `clap`
Token	`src/token.rs`	Token enum definition with `logos` attributes, `LexerExtras`, `LexResult`, `LexerError`
Callbacks	`src/callbacks.rs`	Complex lexing callbacks for strings, characters, newlines, and comments

Module	File	Responsibility
Formatter	`src/formatter.rs`	Converts `LexResult` to output format strings
I/O	`src/io.rs`	File reading, path computation, and output writing
Main	`src/main.rs`	Entry point, orchestrates lexing pipeline

Key interfaces:

```
// Core tokenization function (takes mutable lexer reference)
pub fn tokenize(lex: &mut Lexer<Token>) -> Vec<LexResult>

// Result types
pub struct LexResult {
    pub result: LexResultKind,
    pub line: usize,
    pub col: usize,
}

pub enum LexResultKind {
    Token(Token),
    Error(LexerError),
}

// Lexer state tracking
pub struct LexerExtras {
    pub line: usize,
    pub line_start: usize,
    pub has_token: bool,
    pub saw_comment: bool,
    pub error: Option<LexerError>,  // For callback error pro
pagation
}

// Formatting
pub fn format_lexed_output(results: &[LexResult]) -> String
```

Data flow:

```
Source File (.eta/.eti)
    │
    ▼
read_source_file() ──────► String (source code)
    │
    ▼
Token::lexer_with_extras() ─► Logos lexer instance
    │
    ▼
tokenize() ──────────────────► Vec<LexResult>
    │                           (tokens + positions)
    ▼
format_lexed_output() ──────► String (formatted output)
    │
    ▼
write_lexed_file() ──────────► .lexed file
```

Invariants:

- Line numbers are always ≥ 1 (1-indexed)
- Column numbers are always ≥ 1 (1-indexed)
- `LexerExtras.line_start` always points to the byte offset immediately after the most recent newline
- On error, the returned `Vec<LexResult>` contains all successfully lexed tokens plus exactly one error at the end

4.2 Algorithms and Data Structures

Core algorithms:

The lexer uses `logos`-generated DFA-based matching for simple tokens (keywords, operators, identifiers, integers). For string and character literals, we use custom callback functions that return `Filter<T>` values:

```
lex_string(lexer) -> Filter<String>:
    result = ""
    chars = lexer.remainder().chars()
    while not at closing quote:
        if at EOF or newline:
            lexer.extras.error = UnterminatedLiteral
            return Filter::Emit("")
        if at backslash:
            parse escape sequence (n, ', \\, ", or x{...})
            if invalid escape:
                lexer.extras.error = InvalidEscape
                return Filter::Emit("")
            append escaped char to result
        else:
            append current char to result
    lexer.bump(consumed)
    return Filter::Emit(result)

lex_char(lexer) -> Filter<char>:
    chars = lexer.remainder().chars()
    if empty (closing quote immediately):
        lexer.extras.error = EmptyCharacter
        return Filter::Emit('\\0')
    parse single char (with escape handling)
    if next char is not closing quote:
        if more chars before quote:
            lexer.extras.error = MultiCharacterConstant
        else:
            lexer.extras.error = UnterminatedLiteral
        return Filter::Emit('\\0')
    lexer.bump(consumed)
    return Filter::Emit(char)

tokenize(lexer) -> Vec<LexResult>:
    while let Some(result) = lexer.next():
```

```
        // Check if callback stored an error
        if lexer.extras.error is Some(err):
            push error result and return
        match result:
            Token::Error -> push InvalidCharacter error, retu
rn
            Token::Newline | Token::Comment -> continue (ski
p)
            token -> push token result
    return results
```

Data structures:

- `Token` enum: 45 variants including:
  - `Error` - built-in error token (logos 0.12 style with `#[error]`)
  - `Whitespace`, `Newline`, `Comment` - skipped tokens with callbacks
  - 12 keywords (`if`, `while`, `for`, `let`, `int`, `bool`, `true`, `false`, `use`, `else`, `return`, `length`)
  - 26 operators/symbols (`+`, , , `/`, `%`, `>>`, `<`, `<=`, `>`, `>=`, `==`, `!=`, `=`, `&`, `|`, `!`, `(`, `)`, `[`, `]`, `{`, `}`, `;`, `:`, `,`, `_`)
  - 4 data-carrying variants (`Identifier(String)`, `Integer(u64)`, `String(String)`, `Char(char)`)
- `LexerExtras`: Tracks line number, line start offset, flags for comment/token presence, and `error: Option<LexerError>` for callback error propagation
- `LexerError`: Enum of 6 error types (`InvalidCharacter`, `UnterminatedLiteral`, `InvalidEscape`, `InvalidHex`, `EmptyCharacter`, `MultiCharacterConstant`) with `Default` implementation
- `LexResult`: Combines `LexResultKind` with source position (line, col)

Tradeoffs:

- *Integers as `u64` vs strings:* We chose to parse integers immediately into `u64` values rather than storing raw strings. This catches overflow early but means we don't preserve leading zeros in the original source (not required by spec).

- *Stopping on first error:* Rather than attempting error recovery, we halt on the first error. This simplifies the implementation and avoids cascading spurious errors.
- *Filter-based callbacks:* Using `Filter<T>` callbacks (logos 0.12 style) rather than `Result` allows us to always emit a value while storing errors in `LexerExtras.error`. The `tokenize()` function checks this field after each token and handles errors appropriately.
- *Explicit skip tokens:* `Newline` and `Comment` are explicit enum variants (not truly skipped by logos) so we can run callbacks for line tracking. The `tokenize()` function filters these out with a `continue` statement.

4.3 Implementation Strategy and Collaboration

Implementation strategy: We used a bottom-up approach, implementing and testing individual components before integration:

1. First, defined the `Token` enum with all keywords and operators
2. Added identifier and integer patterns
3. Implemented position tracking via `LexerExtras`
4. Built string/character callbacks with escape handling
5. Created the output formatter
6. Integrated CLI and file I/O
7. End-to-end testing with staff and custom tests

This approach worked well because each layer could be tested independently before integration.

Challenges:

- **Escape sequence parsing:** The `\\x{...}` hex escape required careful handling of variable-length hex digits and proper Unicode code point validation via `char::from_u32()`
- **Position tracking:** Maintaining accurate column positions across multi-byte characters and escape sequences required careful offset arithmetic using `lex.span().start - lex.extras.line_start + 1`

- **Logos 0.12 migration:** The transition to logos 0.12 style with `#[error]` attribute, `Filter<T>` callbacks, and error propagation via `LexerExtras` required reworking our callback architecture
- **Error propagation:** Since `Filter::Emit` must always return a value, we store errors in `LexerExtras.error` and check it in the main `tokenize()` loop after each token

Code reuse: This is our first assignment, so no code was reused from previous work. We did reference the `logos` crate documentation and examples extensively.

---

5. Testing

5.1 Test Plan

We did not assume that the staff-provided PA1 tests were complete, so we designed additional black-box and edge-case tests directly from the Eta lexical specification. The staff tests primarily exercise realistic programs (functions, arrays, control flow) and include a limited number of edge cases (such as the empty character literal and a string containing `\\n` and `\\x{...}`). Our testing strategy supplemented these with targeted tests that stress common lexer failure points: token boundary handling, maximal-munch behavior for multi-character operators, comment handling at end-of-file, and escape processing in string and character literals.

Testing was performed incrementally. As new lexer functionality was implemented (e.g., identifiers, integers, operators, strings, and characters), we added small, focused tests to isolate that behavior before relying on larger integration-style programs. This approach helped localize bugs quickly and ensured that regressions were caught early.

5.2 Test Suite Organization

- **Where tests live:** All PA1 tests are located in `eth/tests/pa1`, with each test consisting of a `.eta` input file and a matching `.lexedsol` expected-output file.
- **How to run tests:**

```
cargo build --release
eth -compilerpath target/release -testpath eth/tests/pa1 e
thScript
```

- **Categories tested:**
  - Keyword–identifier boundaries (e.g., `ifx` is an identifier, not `if` + `x`)
  - Integer–identifier boundaries
  - Identifiers with underscores and single quotes
  - Maximal-munch operator behavior (e.g., `<=` vs `<` `=`)
  - Comments (including comments at EOF without trailing newline)
  - String and character escape handling (`\\n`, `\\\\`, `\\'`, `\\"`, `\\x{...}`)
  - Unterminated string/character literals
  - Invalid escape sequences
  - Empty character literals
  - Multi-character character literals
  - Invalid characters (e.g., `@`, `#`)

5.3 Results

- **Systems tested on:** All tests were run in the provided Linux-based VM environment using the course test harness, as well as locally on macOS during development.
- **Passing tests:** At the time of submission, all staff-provided PA1 tests and all additional custom tests pass.
- **Failing tests:** None.

---

6. Work Plan

The work was divided by major components with clear dependencies:

Team Member	Responsibilities
Charles	Defined shared token data structures (`Token` enum, `LexerExtras`, `LexResult`). Implemented identifiers, keywords, integers, symbols, comments, and source position tracking. All other components depended on this foundation.
Yoseph	Implemented string and character literal handling in `callbacks.rs`, including escape sequence processing (`\\n`, `\\\\`, `\\x{...}`) and lexical error generation. Integrated callbacks with the main lexer loop.
Temi	Handled command-line parsing (`cli.rs`), file I/O operations (`io.rs`), output formatting (`formatter.rs`), and build infrastructure (`etac-build`). Worked on integration once token formats were finalized.
Fadi	Focused on test design, integration testing, documentation (this overview), and edge-case validation throughout development. Enabled incremental testing and early detection of interface mismatches.

This division minimized blocking dependencies: Charles's token definitions were completed first, enabling parallel work on callbacks (Yoseph) and I/O (Temi), while Fadi provided continuous testing feedback.

---

7. AI Usage

We used AI tools (Claude, GitHub Copilot) in the following ways:

- **Debugging assistance:** When encountering confusing `logos` crate behavior, we used AI to explain error messages and suggest fixes
- **Documentation:** AI helped draft initial versions of code comments and this overview document
- **Test brainstorming:** We asked AI to suggest edge cases we might have missed (e.g., "what are common lexer edge cases for escape sequences?")
- **Rust syntax:** Copilot provided autocomplete suggestions for boilerplate code (match arms, struct definitions)

AI was helpful for accelerating routine tasks and catching blind spots in test coverage. However, we found that AI suggestions for the core lexer logic often missed nuances of the Eta specification, so all lexer code was written and verified manually against the spec.

---

8. Known Problems

- **No error recovery:** The lexer stops on the first error encountered. Files with multiple errors will only report the first one.
- **Unicode in identifiers:** Per the Eta spec, identifiers are ASCII-only. Our lexer correctly rejects non-ASCII characters, but the error message ("invalid character") could be more specific.
- **Large integers:** Integers exceeding `u64::MAX` will cause a parse error. The Eta language uses signed 64-bit integers, so values between `i64::MAX + 1` and `u64::MAX` are technically invalid but will lex successfully (semantic checking would catch this in later phases).
- **No `.eti` interface file distinction:** Currently, `.eta` and `.eti` files are lexed identically. Future phases may need to distinguish between them.

---

9. Comments

This assignment was a good introduction to compiler construction. The `logos` crate made the basic lexer straightforward to implement, but the callback system for complex literals required significant experimentation to understand properly.

What was surprising: The amount of edge cases in string/character literal parsing—we initially underestimated how many ways escape sequences can go wrong.

What was hard: Getting position tracking exactly right, especially for multi-line strings and after escape sequences. Off-by-one errors in column numbers were common early on. Additionally, the logos 0.12 `Filter` API required us to rethink error handling—we can't return errors directly from callbacks, so we had to use the `LexerExtras.error` field for propagation.

What we liked: The clear specification format and comprehensive test harness made it easy to verify correctness incrementally. The `strum` crate's `Display` derive made token formatting straightforward.

What we would change: More examples in the handout for edge cases (e.g., what output to produce for `'\\x{}'` or `'\\x{FFFFFFFF}'`) would have saved debugging time. Also, clearer documentation on logos 0.12 vs 0.11 callback conventions would have helped.